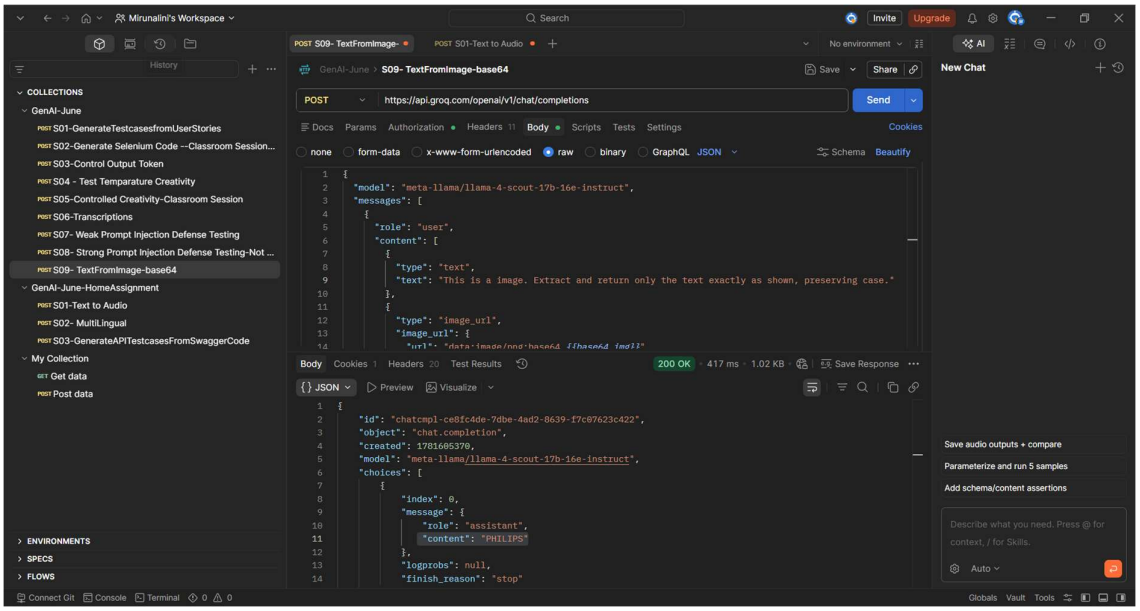
S09 – TextFromImage-base64

```
{
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "This is a image. Extract and return only the text exactly as shown, preserving case."
        },
        {
          "type": "image_url",
          "image_url": {
            "url": "{{base64_img}}"
          }
        }
      ]
    }
  ]
}
```

Output


```
{
    "id": "chatcmpl-ce8fc4de-7dbe-4ad2-8639-f7c07623c422",
    "object": "chat.completion",
    "created": 1781605370,
    "model": "meta-llama/llama-4-scout-17b-16e-instruct",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": "PHILIPS"
            },
            "logprobs": null,
            "finish_reason": "stop"
        }
    ],
    "usage": {
        "queue_time": 0.14169864,
        "prompt_tokens": 177,
        "prompt_time": 0.012228427,
        "completion_tokens": 4,
        "completion_time": 0.009022457,
        "total_tokens": 181,
        "total_time": 0.021250884
```

```
  },
  "usage_breakdown": null,
  "system_fingerprint": "fp_79da0e0073",
  "x_groq": {
    "id": "req_01kv7z9285e4486dqx743kavyr",
    "seed": 1066956823
  },
  "service_tier": "on_demand"
}
```

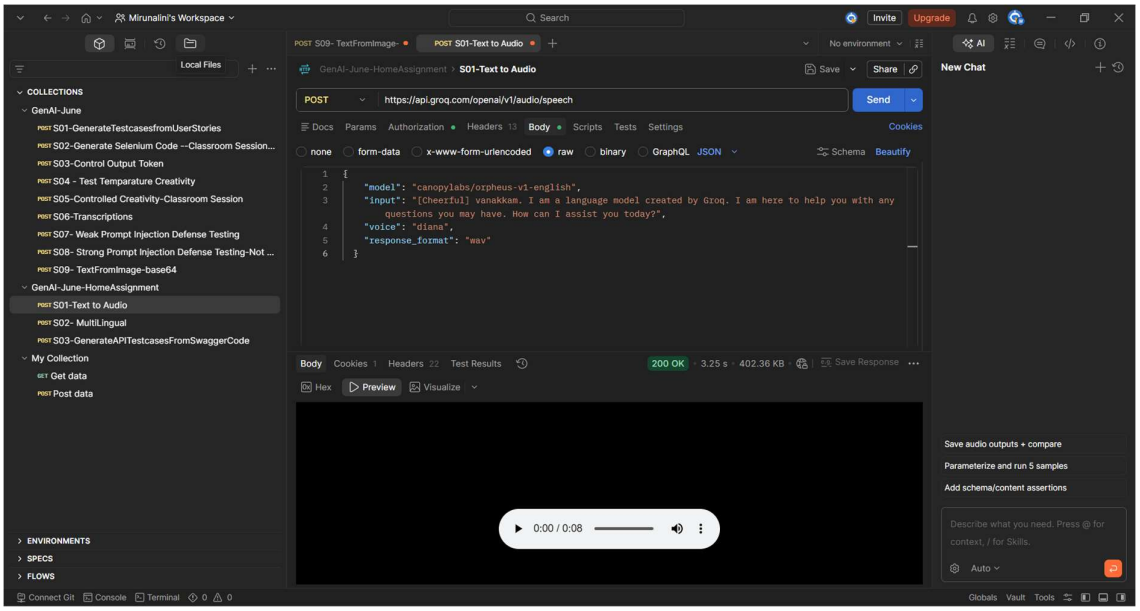



S01-Text to Audio


```
{
  "model": "canopylabs/orpheus-v1-english",
  "input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?",
  "voice": "diana",
  "response_format": "wav"
}
```


Output

response.wav



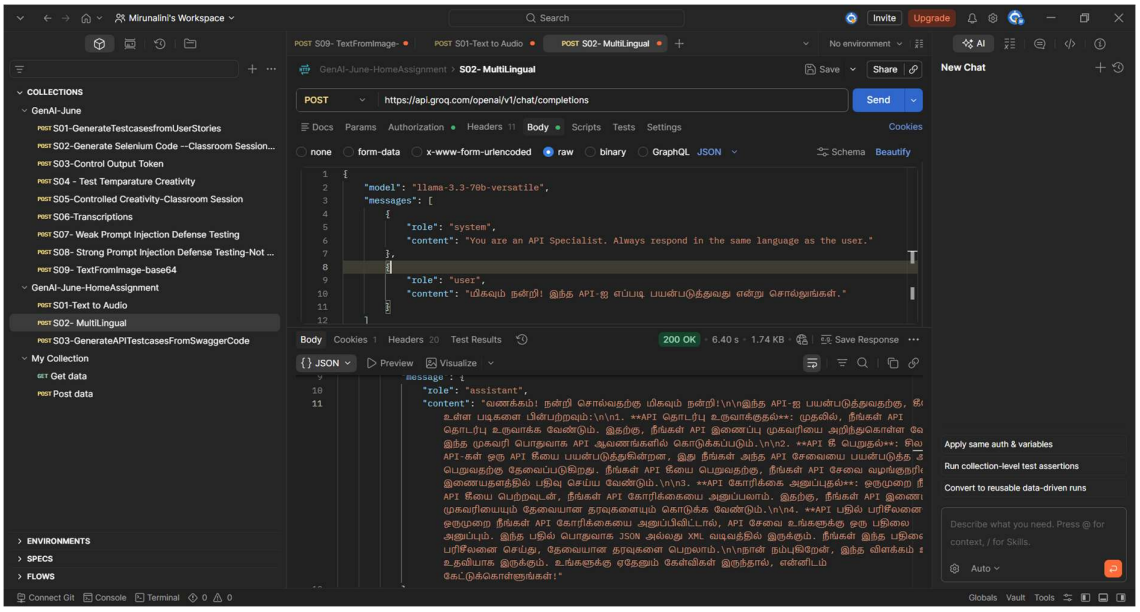
S02- MultiLingual

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as the user."
    },
    {
      "role": "user",
      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்."
    }
  ]
}
```

Output


```
{
  "id": "chatcmpl-b6b6e493-92ab-4261-9f6f-37b450a9cf39",
  "object": "chat.completion",
  "created": 1781619082,
  "model": "llama-3.3-70b-versatile",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
```

"content": "வணக்கம்! நன்றி சொல்வதற்கு மிகவும் நன்றி!\n\nஇந்த API-ஐ பயன்படுத்துவதற்கு, கீழே உள்ள படிகளை பின்பற்றவும்:\n\n1. **API தொடர்பு உருவாக்குதல்**: முதலில், நீங்கள் API தொடர்பு உருவாக்க வேண்டும். இதற்கு, நீங்கள் API இணைப்பு முகவரியை அறிந்துகொள்ள வேண்டும். இந்த முகவரி பொதுவாக API ஆவணங்களில் கொடுக்கப்படும்.\n\n2. **API கீ பெறுதல்**: சில API-கள் ஒரு API கீயை பயன்படுத்துகின்றன, இது நீங்கள் அந்த API சேவையை பயன்படுத்த அனுமதி பெறுவதற்கு தேவைப்படுகிறது. நீங்கள் API கீயை பெறுவதற்கு, நீங்கள் API சேவை வழங்குநரின் இணையதளத்தில் பதிவு செய்ய வேண்டும்.\n\n3. **API கோரிக்கை அனுப்புதல்**: ஒருமுறை நீங்கள் API கீயை பெற்றவுடன், நீங்கள் API கோரிக்கையை அனுப்பலாம். இதற்கு, நீங்கள் API இணைப்பு முகவரியையும் தேவையான தரவுகளையும் கொடுக்க வேண்டும்.\n\n4. **API பதில் பரிசீலனை**: ஒருமுறை நீங்கள் API கோரிக்கையை அனுப்பிவிட்டால், API சேவை உங்களுக்கு ஒரு பதிலை அனுப்பும். இந்த பதில் பொதுவாக JSON அல்லது XML வடிவத்தில் இருக்கும். நீங்கள் இந்த பதிலை பரிசீலனை செய்து, தேவையான தரவுகளை பெறலாம்.\n\nநான் நம்புகிறேன், இந்த விளக்கம் உங்களுக்கு உதவியாக இருக்கும். உங்களுக்கு ஏதேனும் கேள்விகள் இருந்தால், என்னிடம் கேட்டுக்கொள்ளுங்கள்!"
        },
        "logprobs": null,
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "queue_time": 0.054604974,
      "prompt_tokens": 133,
      "prompt_time": 0.007137635,
      "completion_tokens": 1358,
      "completion_time": 5.586271144,
      "total_tokens": 1491,
      "total_time": 5.593408779
    },
    "usage_breakdown": null,
    "system_fingerprint": "fp_dae98b5ecb",
    "x_groq": {
      "id": "req_01kv8cbc1he75tr24t6s49z3x0",
      "seed": 1729586395
    },
    "service_tier": "on_demand"
}



S03-GenerateAPITestcasesFromSwaggerCode

```
{
 "model": "openai/gpt-oss-120b",
 "messages": [
  {
   "role": "system",
   "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
  },
  {
   "role": "user",
   "content": "I — INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2) Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nC — CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nP — PERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nO — OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside Swagger Petstore Pet API\n- Use business-readable method names\n\nTARGET API:\nSwagger Petstore\nResource: Pet\nOperations required:\n- Add a new pet\n- Find pet by ID\n- Update an existing pet\n- Delete a pet"
  }
 ]
}
```

Output

```
{
  "id": "chatcmpl-2a763a7c-ee54-4568-875c-0a737bb7f49f",
  "object": "chat.completion",
  "created": 1781619365,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "import io.restassured.RestAssured;\nimport
io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport
org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport
org.testng.annotations.Test;\n\nimport java.util.*;\n\npublic class PetApiTests {\n\n    private static
long petId;\n\n    @BeforeClass\n    public void setUp() {\n        RestAssured.baseURI =
\"https://petstore.swagger.io/v2\";\n        petId =
System.currentTimeMillis();\n    }\n\n    @Test\n    public void createPet() {\n        Map<String,
Object> pet = new HashMap<>();\n        pet.put(\"id\", petId);\n        pet.put(\"name\",
\"Rex\");\n        pet.put(\"status\", \"available\");\n\n        List<String> photoUrls = new
ArrayList<>();\n        photoUrls.add(\"http://example.com/photo\");\n        pet.put(\"photoUrls\",
photoUrls);\n\n        Map<String, Object> tag = new HashMap<>();\n        tag.put(\"id\",
0);\n        tag.put(\"name\", \"dog\");\n        List<Map<String, Object>> tags = new
ArrayList<>();\n        tags.add(tag);\n        pet.put(\"tags\", tags);\n\n        Response response =
RestAssured\n                .given()\n                .contentType(ContentType.JSON)\n                .body(pet)\n
        .when()\n                .post(\"/pet\")\n                .then()\n                .extract()\n                .response(
);\n\n        Assert.assertEquals(response.getStatusCode(),
200);\n        Assert.assertEquals(response.jsonPath().getLong(\"id\"),
petId);\n        Assert.assertEquals(response.jsonPath().getString(\"name\"),
\"Rex\");\n        Assert.assertEquals(response.jsonPath().getString(\"status\"),
\"available\");\n    }\n\n    @Test(dependsOnMethods = \"createPet\")\n    public void getPetById()
{\n        Response response = RestAssured\n                .given()\n                .pathParam(\"petId\",
petId)\n                .when()\n                .get(\"/pet/{petId}\")\n                .then()\n                .extract()\n
        .response();\n\n        Assert.assertEquals(response.getStatusCode(),
200);\n        Assert.assertEquals(response.jsonPath().getLong(\"id\"),
petId);\n        Assert.assertEquals(response.jsonPath().getString(\"name\"),
\"Rex\");\n        Assert.assertEquals(response.jsonPath().getString(\"status\"),
\"available\");\n    }\n\n    @Test(dependsOnMethods = \"getPetById\")\n    public void updatePet()
{\n        Map<String, Object> updatedPet = new HashMap<>();\n        updatedPet.put(\"id\",
petId);\n        updatedPet.put(\"name\", \"Max\");\n        updatedPet.put(\"status\",
\"available\");\n\n        List<String> photoUrls = new
ArrayList<>();\n        photoUrls.add(\"http://example.com/photo\");\n        updatedPet.put(\"photoUr
ls\", photoUrls);\n\n        Map<String, Object> tag = new HashMap<>();\n        tag.put(\"id\",
0);\n        tag.put(\"name\", \"dog\");\n        List<Map<String, Object>> tags = new
ArrayList<>();\n        tags.add(tag);\n        updatedPet.put(\"tags\", tags);\n\n        Response response
=
RestAssured\n                .given()\n                .contentType(ContentType.JSON)\n                .body(updated
Pet)\n                .when()\n                .put(\"/pet\")\n                .then()\n                .extract()\n                .res
ponse();\n\n        Assert.assertEquals(response.getStatusCode(),
```

200);\n Assert.assertEquals(response.jsonPath().getLong(\"id\"), petId);\n Assert.assertEquals(response.jsonPath().getString(\"name\"), \"Max\");\n Assert.assertEquals(response.jsonPath().getString(\"status\"), \"available\");\n }\n\n @Test(dependsOnMethods = \"updatePet\")\n public void deletePet() {\n Response deleteResponse = RestAssured\n .given()\n .pathParam(\"petId\", petId)\n .when()\n .delete(\"/pet/{petId}\")\n .then()\n .extract()\n .response();\n\n Assert.assertEquals(deleteResponse.getStatusCode(), 200);\n\n Response getAfterDelete = RestAssured\n .given()\n .pathParam(\"petId\", petId)\n .when()\n .get(\"/pet/{petId}\")\n .then()\n .extract()\n .response();\n\n Assert.assertEquals(getAfterDelete.getStatusCode(), 404);\n }\n}",
"reasoning": "We need to output Java code only, no markdown fences, no extra text. Should include imports, class, @BeforeClass, test methods for createPet, getPetById, updatePet, deletePet. Use Rest Assured, TestNG. Include reusable base URI setup. Use assertions for status code and important response fields.\n\nWe'll need to generate a unique pet ID maybe using random. Use static long petId. Use request body as JSON string or Map. Provide assertions: status code, name, status, etc.\n\nWe'll need to set baseURI in @BeforeClass.\n\nWill need to import: io.restassured.RestAssured; io.restassured.http.ContentType; io.restassured.response.Response; org.testng.annotations.*; org.testng.Assert; java.util.*;\n\nWrite class PetApiTests.\n\nMethods: createPet, getPetById, updatePet, deletePet. Use @Test(priority=...). Use dependsOnMethods to enforce order.\n\nWill need to store petId after creation for subsequent tests.\n\nImplementation details:\n\n@BeforeClass public void setup() { RestAssured.baseURI = \"https://petstore.swagger.io/v2\"; }\n\nTest createPet:\n\nMap<String, Object> pet = new HashMap<>();\npet.put(\"id\", petId);\npet.put(\"name\", \"Rex\");\npet.put(\"status\", \"available\");\nList<Map<String, String>> photoUrls = new ArrayList<>();\nphotoUrls.add(Collections.singletonMap(\"url\", \"http://example.com/photo\"));\npet.put(\"photoUrls\", Collections.singletonList(\"http://example.com/photo\"));\nList<Map<String, Object>> tags = new ArrayList<>();\nMap<String, Object> tag = new HashMap<>();\ntag.put(\"id\", 0);\ntag.put(\"name\", \"dog\");\ntags.add(tag);\npet.put(\"tags\", tags);\n\nResponse response = given().contentType(ContentType.JSON).body(pet).when().post(\"/pet\").then().extract().response();\nAssert.assertEquals(response.getStatusCode(), 200);\nAssert.assertEquals(response.jsonPath().getLong(\"id\"), petId);\nAssert.assertEquals(response.jsonPath().getString(\"name\"), \"Rex\");\nAssert.assertEquals(response.jsonPath().getString(\"status\"), \"available\");\n\nStore petId static variable.\n\nTest getPetById dependsOnMethods = \"createPet\".\n\nResponse get = given().pathParam(\"petId\", petId).when().get(\"/pet/{petId}\").then().extract().response();\nAssert.assertEquals(get.getStatusCode(), 200);\nAssert.assertEquals(get.jsonPath().getString(\"name\"), \"Rex\");\n\nTest updatePet dependsOnMethods = \"getPetById\". Change name.\n\nMap<String, Object> updatedPet = new HashMap<>(pet);\nupdatedPet.put(\"name\", \"Max\");\nResponse upd = given().contentType(ContentType.JSON).body(updatedPet).when().put(\"/pet\").then().extract().response();\nAssert.assertEquals(upd.getStatusCode(), 200);\nAssert.assertEquals(upd.jsonPath().getString(\"name\"), \"Max\");\n\nTest deletePet dependsOnMethods = \"updatePet\". Delete.\n\nResponse del = given().pathParam(\"petId\", petId).when().delete(\"/pet/{petId}\").then().extract().response();\nAssert.assertEquals(del.getStatusCode(), 200);\nThen try get again and expect 404.\n\nResponse notFound = given().pathParam(\"petId\",

petId).when().get(\"/pet/{petId}\").then().extract().response();\nAssert.assertEquals(notFound.getStatusCode(), 404);\n\nMake sure to import static io.restassured.RestAssured.*;\n\nBut can use RestAssured.given().\n\nWrite code accordingly. Use long random ID: System.currentTimeMillis().\n\nOk output."
        },
        "logprobs": null,
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "queue_time": 0.846305498,
      "prompt_tokens": 396,
      "prompt_time": 0.01818607,
      "completion_tokens": 1673,
      "completion_time": 3.522804958,
      "total_tokens": 2069,
      "total_time": 3.540991028,
      "completion_tokens_details": {
        "reasoning_tokens": 769
      }
    },
    "usage_breakdown": null,
    "system_fingerprint": "fp_0708ac49a5",
    "x_groq": {
      "id": "req_01kv8cm1mdf56sf06tp4jf2hvx",
      "seed": 1595974380
    },
    "service_tier": "on_demand"
}